Lab 1:

Python Programming - Basics

Description :

This lab introduces basic Python programming concepts using simple exercises. Students will practice variables, user input, arithmetic operations, strings, loops, conditionals, and random numbers.

All exercises are written without functions or recursion to simplify learning.

Exercise 1 : Integer Input and Operations

Write a Python program that, Asks the user to enter two integers and displays their: Sum, Difference, and Product.

Exercise 2 : Counting Vowels in a String

Write a Python program that allows the user to enter a string from the keyboard.

The program should then analyze the entered string and determine the number of vowels it contains. For this exercise, vowels are defined as the characters: a, e, i, o, u

The program must:

- Read the string entered by the user
- Examine each character of the string
- Count how many characters are vowels
- Display the total number of vowels found in the string

  Examples Input: programming

Output: Number of vowels = 3

Exercise 3 : Distance Between Two Points

Write a Python program that calculates and displays the distance with two decimal places between two points A and B in the Cartesian plane, whose coordinates (X_A, Y_A) and (X_B, Y_B) are entered from the keyboard as integers. The formula for calculating the distance is as follows:

distance $=\sqrt{(X_A - X_B)^2} + (Y_A - Y_B)^2$

Exercise 4: Factorial of a Number

Write a Python program that takes a user input of an integer and calculates and prints the factorial of that number. For example, if the user enters the number 5, the program returns: fact(5) = 120.

Exercise 5 : Days in a Month

Write a Python program that asks the user to enter the number of a month, an integer between 1 and 12, and then displays the number of days in that month.

For the case of February, also ask for the **year** as an integer between 1900 and 2024. If it is a leap year (divisible by 4 and not divisible by 100, or divisible by 400), the number of days is 29; otherwise, it is 28.

Exercice 6: Palindrome number

Write a Python program that asks the user to enter an integer N strictly greater than 2.
The program must then check whether this number is a **palindrome**.
A number is said to be a palindrome if it reads the same from left to right and from right to left.
In other words, reversing the digits of the number gives exactly the same number.
The program should:

- Read the integer entered by the user
- Reverse the digits of the number
- Compare the original number with its reversed value
- Display an appropriate message indicating whether the number is a palindrome or not

Examples :

Input: 121

Output: Palindrome

Input : 122

Output : Not a palindrome

Exercice 7: Niven number

Write a Python program that asks the user to enter an integer N strictly greater than 2. The program must then check whether this number is a Niven number.

A number is said to be a Niven number if it is divisible by the sum of its digits.

The program should:

- Read the integer entered by the user
- Extract and sum all the digits of the number
- Test whether the number is divisible by this sum

Display an appropriate message indicating whether the number is a Niven number or not.

Example : Input: 18

Sum of digits: 1 + 8 = 9

18 is divisible by 9

Output: Niven number

Exercise 8 : Quadratic Equation Roots

We want to create a Python program that calculates the roots of a quadratic equation $ax^2 + bx + c$, where a, b, and c are three real numbers.

The program should:

- Ask the user to input the coefficients of the quadratic equation as 3 floating-point numbers (float), stored in variables a, b, and c.
- Calculate the discriminant of the quadratic equation, named delta.
- Test the value of the discriminant. Depending on the sign of the discriminant, display a message indicating the number and type of roots (real or complex) of the quadratic equation, and then display the values of these roots.

Exercise 9: Prime factor

Write a Python program that allows the user to enter an integer N>2 and displays its **prime factors**, separated by spaces.
A number is considered prime if it is divisible only by 1 and itself.
Example:
Input: N = 150
Output: 2 3 5

Exercise 10:

Part 1:
Write a Python script that allows the user to enter a series of **three-digit positive numbers**, one by one, in order to calculate and display:

- The maximum value, named maxnb
- The minimum value, named minnb
- The average of the numbers, named moynb, with 3 decimal places

The input stops when the user enters 0.

Part 2:
Write a Python script that displays all Keith numbers between minnb and maxnb.
A three-digit Keith number is a natural number with three digits that appears as a term in a linear recurrence sequence of order 3, where each term is obtained by summing the three preceding terms.

<u>Example:</u> The sequence of the number 742 is given by:

$U_0=7$
$U_1=4$
$U_2=2$
$U_{n+3}=U_{n+2} + U_{n+1} + U_n$

The terms of this sequence are:

7, 4, 2, 13, 19, 34, 66, 119, 219, 404, 742 → So, 742 is a Keith number.

Exercice 11:

Write a Python program that simulates the **mystery number game**. In this game, the user plays against the computer following these rules:

- The computer randomly selects an integer between **1 and 100**.
- The user has **at most 7 attempts** to guess the mystery number.
- After each guess, the program provides a hint:
    - "Too high" if the guessed number is greater than the mystery number

- "Too low" if the guessed number is smaller than the mystery number

If the user finds the mystery number within the allowed number of attempts, the program should display: